Worker Isolation — Responsibility / Resource Table

Branch `fix/worker-isolation` (`cdc56a6` off `35884ee`). Feature gated by `WORKER_ISOLATION_ENABLED` (default **off** → byte-for-byte `35884ee` in-process path). One `child_process.fork` per job attempt: the **parent** stays a responsive BullMQ supervisor; the **child** runs the existing `runAnalysisPipeline` unchanged. This table is the pre-implementation contract; the mechanisms below are the ones actually implemented (file:line references are load-bearing).

Concern	Parent (responsive supervisor — never runs pipeline CPU)	Child (heavy processing — owns no lock)	Enforcement (code)
Claim owner	Sole owner of the BullMQ claim, attempt token, heartbeat and **lock renewal**. Never executes pipeline synchronous work, so the event loop that renews the lock is never blocked.	Never touches BullMQ/Redis. Receives only a copy of the claim's lifecycle identity (token + deadline) for fencing its own writes.	`job-isolation.ts` runs in the worker loop; child statically imports no redis/bull. Claim/renewal unchanged from `35884ee`.
Context propagation	Sends one `init` IPC message: `opts` (minus the non-serialisable `cancelCheck`), lifecycle `{jobId,batchId,attemptNumber,deadlineAt,attemptToken,revokedReason}`, `attemptId/companyId/frameworkId`, and the current `cancelled` flag.	Rebuilds the attempt lifecycle with the **same token + deadline** via `createAttemptLifecycle`; re-establishes diag-context and llm-usage context around the pipeline.	`job-isolation.ts` `message:ready→init`; `job-child.ts` `run()` + `runWithLlmContext` / `runWithDiagContext`.
Authoritative transitions	Owns **all** terminal state: BullMQ completion/failure, every DB terminal write, and `revokeAttempt`. Child results are advisory until the parent applies them.	Reports `{t:result}` / `{t:error}` only; forwards `revokedReason` if it self-revoked. Writes no BullMQ/terminal state.	`job-isolation.ts` `case "result"` → `revokeAttempt(lc,…)` then `resolve`; `case "error"` → `reject(rebuildError)`.
Cancellation	Polls `opts.cancelCheck()` every 500 ms → `{t:cancel}`. Lifecycle abort (watchdog / ownership lost)	`cancelFlag` / `revoke` flip the child's `cancelCheck` (`cancelFlag || lc.revoked`); pipeline observes it at the	`job-isolation.ts` `cancelPoll`, `onRevoke`; `job-child.ts` `process.on("message")`.

Concern	Parent (responsive supervisor — never runs pipeline CPU)	Child (heavy processing — owns no lock)	Enforcement (code)
	→ `{t:revoke}`. Both also arm termination.	same checkpoints as in-process.	
Child / descendant termination	Child forked `detached:true` (own process group). On deadline/revoke/cancel: SIGTERM to the **process group** after `WORKER_ISOLATION_KILL_GRACE_MS` (10 s), SIGKILL after +5 s. On any exit, `process.kill(-pid, SIGKILL)` reaps stragglers; `process.on("exit")` SIGKILLs all live children.	Chromium / pdftotext descendants inherit the group and die with it — no orphans.	`job-isolation.ts` `killGroup`, `armKill`, `exit` reaper, `exit` hook.
Retries	A blocked/crashed/killed child surfaces as a **retriable** error (`"Job watchdog timeout: … after <reason>"` when killed; raw exit otherwise). `ProviderScoringError` / `LifecycleCancelledError` are reconstructed faithfully so existing retry/classification logic is unchanged.	Crash while holding permits → parent still releases them (below) before the error is classified.	`job-isolation.ts` `exit` → `reject`; `rebuildError`; retry policy untouched from `35884ee`.
Aggregate DB budget	Reserves `WORKER_ISOLATION_PARENT_PG_RESERVE=4` connections; sets each child's `PG_POOL_MAX = floor((20 − 4) / WORKER_CONCURRENCY)`	Opens its own small pool capped at the budgeted 4.	`db.ts` `PG_PARENT_RESERVE` / `PG_POOL_MAX`; `job-isolation.ts` `childPgBudget` (refuses to spawn if budget < 1).

Concern	Parent (responsive supervisor — never runs pipeline CPU)	Child (heavy processing — owns no lock)	Enforcement (code)
	= **4** at the deployed concurrency (4). Replica-wide total = 4 + 4×4 = **20 = PG_POOL_MAX — not multiplied**.		
Aggregate LLM budget	Owns the single `llmSlotBroker` for the replica (`LLM_MAX_CONCURRENCY=8`); children `acquire` / `release` **over IPC**; a crash releases the child's outstanding grants.	Calls the same `releaseLlmSlot()` /acquire call sites; in a child these are brokered, so the cap stays **per-replica (8)**, never per-child.	`ai-providers.ts` `llmSlotBroker` ; `isolation-ipc.ts` `remoteAcquire/remoteRelease` ; `job-isolation.ts` `acquire/release/releaseAll` .
Aggregate browser / subproc budget	Owns `processorSlotBroker.browser` (`MAX_CONCURRENT_BROWSER=1`) and `.proc` (`MAX_CONCURRENT_SUBPROC=6`); brokers both over IPC identically.	Same call sites, brokered → caps stay **per-replica (1 / 6)**, never per-child.	`processor.ts` `processorSlotBroker` ; `isolation-ipc.ts` ; `job-isolation.ts` `releaseAll` .

Preserved unchanged: research/scoring code (child runs existing `runAnalysisPipeline`), scoring semantics, time budgets (`deadlineAt`), and intended concurrency (`WORKER_CONCURRENCY`). Per-process pools and permits are **partitioned/brokered, never multiplied** across children. No renewal-only workaround: renewal stays with the parent *because* the parent no longer runs blocking CPU — isolation is the fix, not a longer lock.